

ENGN 0030 Wearable Sensor Guide: Impact Sensor

Brief Description of Project

The aim of this project is an impact detector built with an Arduino. Wearable impact sensors have applications in elderly care, athletics, and workplace safety. This project uses an IMU sensor, MPU-6050, a 6-axis accelerometer and gyroscope module, together with a buzzer and an LCD screen to detect and alert on hard impacts.

What your finished project will look like:

- The MPU-6050 module continuously measures acceleration.
- A buzzer sounds when a hard impact is detected.
- An LCD screen displays “Monitoring..” during normal use and “IMPACT DETECTED” when an impact occurs, or an LED that flashes on impact.

****See appendix for some relevant resources.*

****Please reference the Arduino Syntax sheet for further guidance on C++ syntax!!*

Stakeholders Discussion

- Elderly care / fall detection for independent living
- Athletics and sports safety
- Workplace safety (construction, warehouse environments)

More on MPU-6050: Accelerometer and Gyroscope Sensor

The MPU-6050 is an I2C based motion sensor with applications in wearables, robotics, and biomedical devices. This module measures acceleration along three axes (X, Y, Z) as well as rotational velocity (gyroscope), though this project focuses only on the accelerometer.

How exactly does the MPU-6050 detect an impact?

The accelerometer measures acceleration along the X, Y, and Z axes, each reported in units of g ($1g = 9.81 \text{ m/s}^2$, roughly the pull of gravity at rest). Because the sensor could be mounted in any orientation, the three axis readings are combined into a single magnitude value using the Pythagorean formula:

$$\text{magnitude} = \sqrt{x^2 + y^2 + z^2}$$

At rest, this magnitude reads close to $1g$. During a hard impact, it spikes sharply well above that baseline for a brief moment. Comparing each reading against a chosen threshold allows the Arduino to detect when an impact has occurred.

Materials List

- Arduino UNO R3
- LCD 1602 (optional)
- Buzzer
- MPU-6050 Module (GY-521) + 4-pin header
- Jumper Wires
- Breadboard
- Male A/B USB cord
- 220 ohm resistor (for buzzer, optional)
- Potentiometer (10k), if using a non-I2C LCD
- LEDs (optional)

Building Guidelines

Soldering the MPU-6050 Sensor

Before beginning:

The BDW electronics workshop must be completed before soldering. This can be done either through the ENGN 0030 BDW TAs or with the BDW monitors. The workshop teaches safe handling of soldering equipment and proper technique.

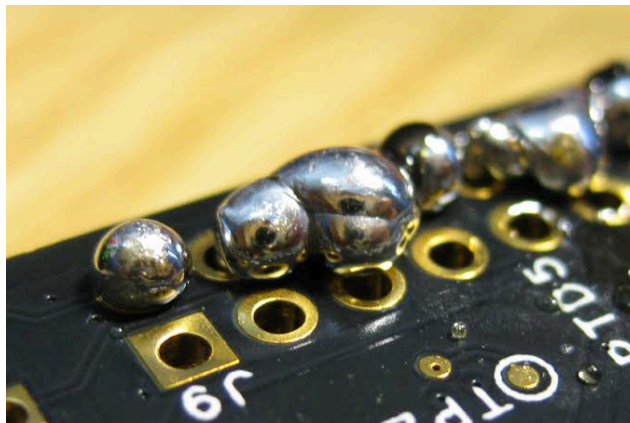
Why soldering matters

The MPU-6050 sensor comes with loose pins that need to be firmly attached.

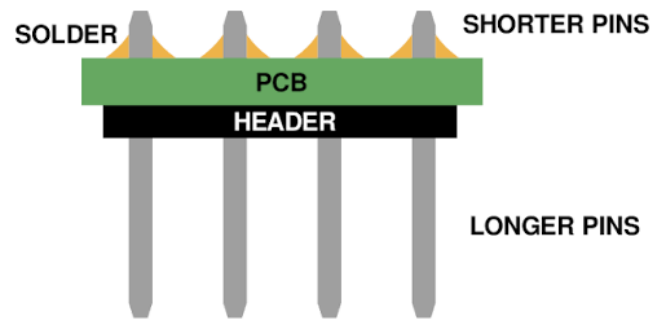
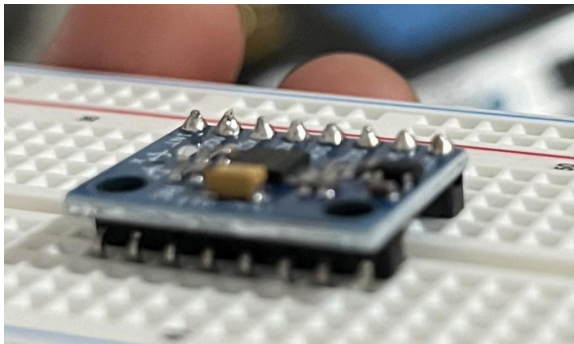
Without soldering:

- The pins may shift or disconnect during testing.
- Electrical connections may be unreliable, leading to inconsistent readings.
- A hard impact (the exact event this project is designed to detect) can jostle a loose connection and cause the sensor to freeze mid-test.

Soldering the pins ensures a secure, stable connection and protects the sensor from damage.



Incorrect Soldering: Too much solder



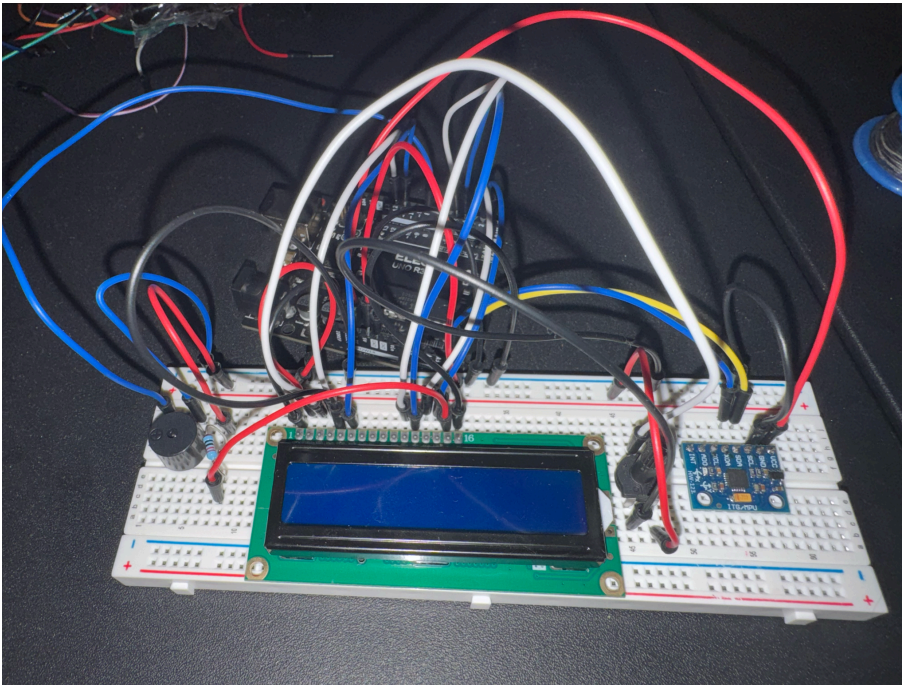
Correct Soldering

Basic soldering steps

1. **Prepare your materials** – Gather the heart rate sensor, the 4-pin header, soldering iron, solder, and a helping-hand or clamp to hold the parts in place.
2. **Insert the pins** – Place the 4-pin header into the sensor board so that the short ends stick out from the topside.
3. **Secure the board** – Use a clamp or helping-hand tool to hold the sensor steady
4. **Heat the joint** – Tin your iron, then touch the tip of the soldering iron to both the metal pin and the board pad for a second or two.
5. **Apply solder** – Feed a small amount of solder into the joint (not directly onto the iron). It should flow smoothly around the pin and pad.
6. **Remove heat** – Pull away the solder first, then the iron. Let the joint cool naturally (don't blow on it).
7. **Repeat for all 4 pins** – Make sure each joint is shiny and conical in shape, not blobby or dull.

Once all four pins are soldered, double-check that no solder bridges are connecting neighboring pins (like in the incorrect example). This can cause shorts and damage the sensor.

Breadboard Wiring



*** Make another version without the LCD so it is easier to see?

1. Gather and prepare all materials.
 1. Install Arduino IDE software.
 2. Add the appropriate libraries to Arduino IDE. (see Libraries)
2. Connect the pin header to the MPU-6050 and attach it to the breadboard. (The header pins should be soldered rather than left as a loose friction fit, see Soldering section. This can be skipped for now, but should be done before attempting to use the sensor.)
3. Wire the IMU module to the Arduino. Use jumper wires to connect:
 1. GND - GND
 2. VCC - 3.3V (do NOT use 5V unless the board has an onboard regulator)
 3. SCL - A5
 4. SDA - A4
4. Optional: At this point, check if the MPU-6050 is functioning correctly. A great way to troubleshoot errors with the sensor before adding anything else is to plot acceleration magnitude on Arduino IDE's serial plotter.
 1. Use the code under the section "MPU-6050 Test Code."
 2. Open a new file in Arduino IDE and upload the test code.
 3. Open the serial plotter (Tools > Serial Plotter). Make sure the dropdown is set to 115200 baud.

4. Gently tap the sensor and confirm a spike appears in the plotted magnitude.
5. Wire the buzzer:
 1. Connect the buzzer's signal leg to digital pin 8 on the Arduino.
 2. Connect the buzzer's other leg to GND.
 3. Optional: place a 220 ohm resistor in series with the signal leg to reduce volume.
6. Wire the potentiometer (allows the impact threshold to be tuned live, without reprogramming):
 1. One outer leg -> 5V
 2. Other outer leg -> GND
 3. Middle leg (wiper) -> A0
7. Add a visual indicator for impact detection. Choose ONE of the following options:

Option A: LED (simpler, fewer wires, recommended for a compact build)

1. Anode (long leg) -> one leg of a 220 ohm resistor
2. Other leg of the resistor -> digital pin 7
3. Cathode (short leg) -> GND

Option B: LCD (16x2, Can show live numeric feedback instead of just on/off)

1. Plug the LCD screen into the breadboard.
2. Follow the pin connections for the LCD screen:
 1. VSS -> Ground
 2. VDD -> Power
 3. VO -> Middle pin of a **second, separate** potentiometer used only for contrast (its outer legs go to power and ground; this is not the same potentiometer as Step 6, which controls the threshold)
 4. RS -> 12
 5. RW -> Ground
 6. E -> 11
 7. D4 -> 5
 8. D5 -> 4
 9. D6 -> 3
 10. D7 -> 2
 11. A -> to power through a 220 ohm resistor
 12. K -> Ground

8. The impact sensor hardware build is now complete. The rest of the project is composed of writing/troubleshooting code. Refer to the section "Partial Code Template" for starting code and instructions.

Arduino IDE: Program Your Microcontroller

Libraries

All libraries for this project can be found on Arduino IDE. They can also be downloaded from the Arduino Website.

- Adafruit MPU6050: An Arduino library for the MPU-6050 accelerometer/gyroscope
- Adafruit Unified Sensor: Required dependency for the Adafruit MPU6050 library
- LiquidCrystal: Built into the Arduino IDE, used for the optional LCD display

MPU-6050 Test Code (Use for Step 4: This is not the main project!!!)

```
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <Wire.h>

Adafruit_MPU6050 mpu;

void setup() {
  Serial.begin(115200);

  if (!mpu.begin()) {
    Serial.println("Failed to find MPU6050 chip");
    while (1) delay(10);
  }

  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
  Serial.println("MPU6050 Found!");
}

void loop() {
  sensors_event_t a, g, temp;
  mpu.getEvent(&a, &g, &temp);

  float ax_g = a.acceleration.x / 9.81;
  float ay_g = a.acceleration.y / 9.81;
  float az_g = a.acceleration.z / 9.81;
  float magnitude = sqrt(ax_g * ax_g + ay_g * ay_g + az_g * az_g);

  Serial.println(magnitude); // Send raw data to plotter

  delay(20);
}
```


Partial Code Template

Think of this template as a starting point: feel free to innovate and adapt as needed.

*****Please reference the Arduino Syntax sheet for further guidance on C++ syntax!!**

This page is extremely helpful for understanding Arduino Programming Language:

<https://www.arduino.cc/reference/en/>

Some commands that might be useful in the loop function, for controlling what displays on the LCD screen:

```
lcd.clear();  
lcd.setCursor(0,0);  
lcd.print("text");
```

For the buzzer, keep these commands in mind:

```
tone(8,1000); // 8 = pin on arduino, 1000 = frequency  
delay(100);  
noTone(8);
```

Where to start:

Include necessary library files:

Syntax: `#include<library file>`

Files:

```
#include <Adafruit_MPU6050.h> // For the MPU-6050 sensor  
#include <Adafruit_Sensor.h> // Required dependency  
#include <Wire.h> // For I2C communication  
#include <LiquidCrystal.h> // For the LCD display
```

Declare variables and datatypes:

```
Adafruit_MPU6050 mpu;  
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);  
  
const int BUZZER_PIN = 8;  
  
// Settings you can tune  
const int BEEP_DURATION_MS = _;  
const float IMPACT_THRESHOLD = _; // g's, adjust based on testing  
const unsigned long SAMPLE_INTERVAL_MS = _; // how often to take a new reading  
const unsigned long ALERT_DISPLAY_MS = _; // how long the alert stays on screen  
  
// Frozen sensor detection (a known I2C glitch that can follow a hard impact)  
const int ZERO_STUCK_LIMIT = 5; // how many 0.00 readings in a row counts as frozen  
int zeroCount = 0;  
  
// State  
bool alertActive = false;  
unsigned long alertStartedAt = 0;
```

```
unsigned long lastSampleAt = 0;
```

Write a helper function to initialize (or re-initialize) the sensor. This gets called once in `setup()`, and again any time the sensor is later found to be frozen:

```
void initSensor() {
  Serial.println("Initializing MPU6050...");
  Wire.end();
  delay(100);
  Wire.begin();
  delay(100);

  if (!mpu.begin()) {
    Serial.println("Failed to find MPU6050 chip");
    return;
  }

  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
  Serial.println("MPU6050 ready.");
}
```

Set up the void `setup()` function:

- Open the Serial Monitor
- Set the buzzer pin mode
- Call `initSensor()`
- Start the LCD display and print an initial “Monitoring...” message

```
void setup() {
  // Fill in with baud rate
  Serial.begin(__);

  pinMode(BUZZER_PIN, OUTPUT);

  initSensor();

  lcd.begin(16, 2);
  lcd.print("Monitoring...");
}
```

Set up the void `loop()` function. For the rest of the code, fill in lines with the appropriate commands.

```
void loop() {
  unsigned long now = millis();

  //Only take a new reading every SAMPLE_INTERVAL_MS - otherwise, exit the loop early
  if (__) return;
  lastSampleAt = now;

  //If an alert is currently showing, just wait for it to time out, then reset the LCD and exit
  if (alertActive) {
```



```

    if (now - alertStartedAt > ALERT_DISPLAY_MS) {
        alertActive = false;
        //Clear the LCD
        //Print "Monitoring..." on LCD
    }
    return;
}

//Reading the sensor to know current acceleration
sensors_event_t a, g, temp;
mpu.getEvent(&a, &g, &temp);

//Convert acceleration (m/s^2) to g's by dividing by 9.81
float ax_g = ____;
float ay_g = ____;
float az_g = ____;

//Combine the three axes into one magnitude value
//HINT: magnitude = sqrt(ax_g^2 + ay_g^2 + az_g^2)
float magnitude = ____;

//Check whether the sensor has frozen: a resting reading should be close to
//1.00g, so several exact 0.00 readings in a row means the I2C connection
//glitched and the sensor needs to be reinitialized
if (magnitude == 0.00) {
    zeroCount++;
    if (zeroCount >= ZERO_STUCK_LIMIT) {
        Serial.println("Sensor appears frozen, reinitializing...");
        //Call the function that reinitializes the sensor
        zeroCount = 0;
        return;
    }
} else {
    zeroCount = 0;
}

//Print the magnitude value on Serial Monitor

//If magnitude is greater than IMPACT_THRESHOLD, an impact has occurred
if (____) {
    //Call the function that handles what happens on impact
}
}

```

Write a helper function for what should happen when an impact is detected. It needs to: mark the alert as active and record when it started, update the LCD, and sound the buzzer.

```

void triggerAlert() {
    alertActive = true;
    alertStartedAt = ____;

    //Clear the LCD
    //Set cursor to (0,0)
    //Print "IMPACT DETECTED" on LCD

    //Sound the buzzer using tone(), then a short delay, then sound it again
    //for a more noticeable double-beep

    Serial.println("IMPACT DETECTED");
}

```

Tuning your sensor (Reducing False positives/negatives)

A single threshold check will occasionally trigger on ordinary movement rather than a genuine hard impact. As a goal, consider:

- Raising or lowering `IMPACT_THRESHOLD` based on testing with the specific mounting location.
- Adding a short “lockout” period after an alert so a single impact does not trigger repeated alerts.
- Developing a different, more accurate algorithm

Troubleshooting

- Use the correct connections on the MPU-6050! Note down the orientation of the module somewhere.
- Make sure Arduino IDE is registering the correct port.
- If the sensor reading freezes at exactly 0.00 after a hard impact, this is a known I2C communication glitch. Add code that detects several repeated 0.00 readings in a row and reinitializes the sensor (`Wire.begin()` and `mpu.begin()` again), rather than requiring a manual re-upload.
- The sensor needs to be in a relatively stable, securely mounted position to work consistently – loose header pins are a common cause of unreliable readings.

What to do when getting “avrdude: stk500_recv(): programmer is not responding” when trying to upload a sketch:

1. Check Connections: Ensure the Arduino is properly connected to the computer.
2. Select the Correct Board and Port:
3. In the Arduino IDE, go to Tools > Board and select the correct board model.
4. Go to Tools > Port and make sure the correct COM port is selected. If unsure, unplug the Arduino and see which port disappears, then plug it back in to see which one appears.
5. Reset the Arduino: Press the reset button on the Arduino right before hitting upload in the IDE. This can help establish a connection.
6. Update Drivers: On Windows, ensure the correct drivers are installed for the board.
7. Try a Different USB Port: Switching the USB port on the computer can sometimes help.
8. Check for Other Software: Ensure no other programs (like serial monitors or other IDEs) are using the same COM port.

9. Board Issues: If nothing else works, try uploading a simple sketch (like the Blink example) to see if the problem persists.

Possible Improvements/Customization

- Distinguish a fall from a generic impact by requiring a brief period of near-weightlessness (free-fall) immediately before the impact spike.
- Add Bluetooth (e.g., an HC-05 or HM-10 module) to send an alert to a phone or computer instead of, or in addition to, the onboard buzzer.
- 3D print or otherwise build a small enclosure to make the device wearable on a wrist or waistband.
- Log magnitude data to an SD card for later review and threshold tuning without needing a live USB connection.
- Find other uses for the IMU sensor

Appendix

Arduino UNO R3 pinout diagram: see course Arduino Bootcamp materials.

MPU-6050 pinout: GND, SCL, SDA, VIN, INT, XDA, XCL, AD0 – see the Adafruit MPU6050 product page or datasheet for the full pin reference.

Some Relevant Readings:

https://en.wikipedia.org/wiki/Inertial_measurement_unit

<https://docs.arduino.cc/hardware/uno-rev3/>

<https://en.wikipedia.org/wiki/Microcontroller>

<https://learn.sparkfun.com/tutorials/how-to-use-a-breadboard/all>

https://www.seeedstudio.com/blog/2020/01/17/what-is-imu-sensor-overview-with-arduino-usage-guide/?srsltid=AU7gw4U6af6Q5gYPcd_1AXRF3jajzn18gYdx9ZqWdxNy6MrvbzxAttoH

<https://www.arduino.cc/reference/en/>